# T1: C# y .NET- Elementos básicos de un programa

## Contenido

T1: C# y .NET- Elementos básicos de un programa ..... 1
.Net ..... 2
CLR ..... 2
Assembly ..... 2
CIL ..... 2
El ambiente de desarrollo ..... 3
Cómo crear nuestra primera aplicación ..... 4
La vista de clases ..... 7
Comentarios en el programa ..... 8
Las Variables ..... 8
Conversión de variables ..... 10

# .Net

En la búsqueda de descubrir que es .NET nos encontramos con las primeras definiciones que indican “.NET es un framework de código abierto para crear aplicaciones de escritorio, web y móviles que se pueden ejecutar de forma nativa en cualquier sistema operativo”. Punto NET incluye herramientas, bibliotecas y lenguajes que admiten el desarrollo de software moderno, escalable y de alto rendimiento.

Lo primeros que debemos saber es ¿Que es un Framework? Podemos decir rápidamente que un framework es un marco o esquema de trabajo utilizado por programadores para realizar el desarrollo de software. Profundizando un framework es la composición estructurada de tecnologías que sirve para el desarrollo y organización de un software o aplicación. Se puede componer de bibliotecas, programas y lenguajes, así como herramientas necesarias para el proyecto. Microsoft .NET es una colección de diferentes plataformas de software de Microsoft. El framework original fue desarrollado como una competencia directa a la plataforma Java. Así, los entornos de aplicación pueden ser desarrollados y ejecutados en base a .NET, el entorno .NET ha cambiado significativamente a lo largo de los años y ha ganado en importancia.

El Framework de .NET brinda grandes beneficios no solamente al desarrollador, sino también al proceso de desarrollo. En primer lugar, .NET permite trabajar con código ya existente. Podemos hacer uso de los componentes COM, e incluso, si lo necesitáramos usar, el API de Windows y es multilenguaje. Los lenguajes que hacen uso de .NET comparten las librerías de .NET, por lo que no importa en qué lenguaje programemos, las reconocemos en cualquiera. Esto hace que un programador de C# puede entender fácilmente el código de un programador de Visual Basic .NET y ambos pueden programar en el lenguaje con el que se sientan más cómodos.

Esto se debe a que todos A continuación, conoceremos los diferentes componentes de .NET: CLR, assembly y CIL.

## CLR

El primer componente de .NET que conoceremos es el Common Language Runtime, también denominado CLR. Este es un programa de ejecución común a todos los lenguajes. Este programa se encarga de leer el código generado por el compilador y empieza su ejecución. Sin importar si el programa fue creado con C#, con Visual Basic .NET o algún otro lenguaje de .NET, el CLR lo lee y ejecuta.

## Assembly

Cuando tenemos un programa escrito en un lenguaje de .NET y lo compilamos se genera el assembly. El assembly contiene el programa compilado en lo que conocemos como CIL y también información sobre todos los tipos que se utilizan en el programa.

## CIL

Los programas de .NET no se compilan directamente en código ensamblador del compilador, sino que son compilados a un lenguaje intermedio conocido como CIL. Este lenguaje es leído y ejecutado por el runtime. El uso del CIL y el runtime es lo que le da a .NET su gran flexibilidad y su capacidad de ser multiplataforma.

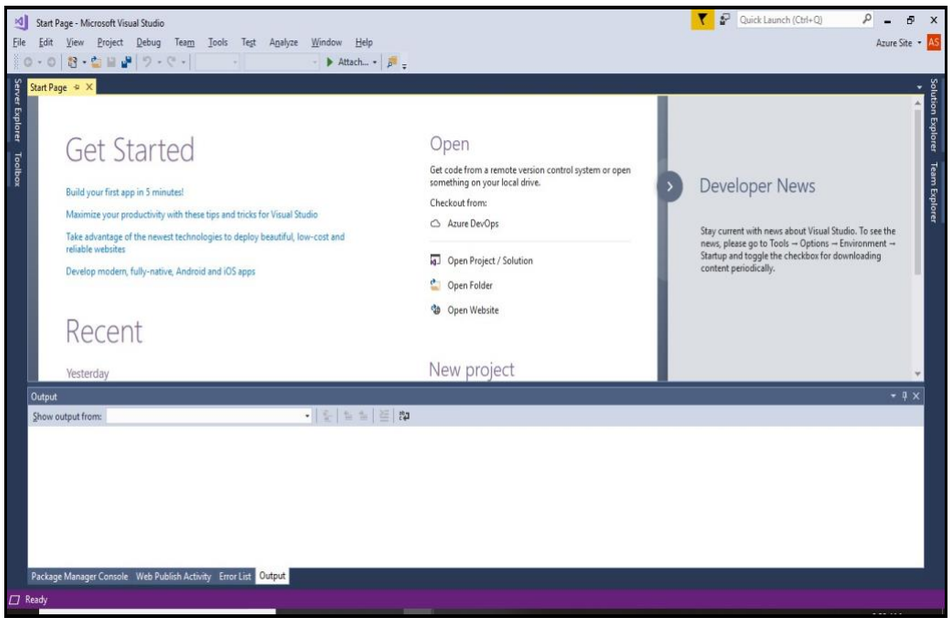
El Framework de .NET tiene lo que se conoce como las especificaciones comunes de lenguaje o CLS por sus siglas en inglés, las cuales son las guías que cualquier lenguaje que desee usar .NET debe de cumplir para poder trabajar con el runtime. Una ventaja de esto es que si nuestro código cumple con las CLS podemos tener interoperabilidad con otros lenguajes, por ejemplo, es posible crear una librería en C# y un programador de Visual Basic .NET puede utilizarla sin ningún problema.

Uno de los puntos más importantes de estas guías es el CTS o sistema de tipos comunes. En los lenguajes de programación, cuando deseamos guardar información, esta se coloca en una variable. Las variables van a tener un tipo dependiendo de la información a guardar; por ejemplo, el tipo puede ser para guardar un número entero, otro para guardar un número con decimales y otro para guardar una frase o palabra. El problema con esto es que cada lenguaje guarda la información de manera diferente, algunos lenguajes guardan los enteros con 16 bits de memoria y otros con 32 bits; incluso algunos lenguajes como C y C++ no tienen un tipo para guardar las cadenas o frases.

Para solucionar esto el Framework de .NET define por medio del CTS cómo van a funcionar los tipos en su entorno. Cualquier lenguaje que trabaje con .NET debe de usar los tipos tal y como se señalan en el CTS.

# El ambiente de desarrollo

Una vez iniciada la aplicación veremos una ventana, como la que aparece a continuación, que muestra la interfaz de uso de la aplicación.




Básicamente la interfaz está dividida en dos partes. En el centro tenemos la página de inicio; aquí encontramos enlaces a nuestros proyectos más recientes, pero también podemos recibir comunicados y noticias sobre el desarrollo de software usando C#. En esta misma parte de la interfaz, pero como páginas diferentes, tendremos las zonas de edición. Estas zonas de edición nos permitirán editar el código del programa, editar la interfaz de usuario, iconos y demás recursos.

Editar el código del programa es una actividad muy sencilla que no debe preocuparnos. Si sabemos utilizar cualquier tipo de editor de textos como Microsoft Word o el Bloc de notas de Windows, entonces podemos editar el código del programa. Los demás editores también son bastante amigables para el usuario, pero no los necesitaremos para realizar los ejemplos mostrados en este libro.

Del lado derecho tenemos una ventana que se conoce como Explorador de soluciones; en esta misma área aparecerán otras ventanas que nos dan información sobre el proyecto que estamos desarrollando. Veremos los detalles mostrados por algunas de estas ventanas un poco más adelante en este libro.

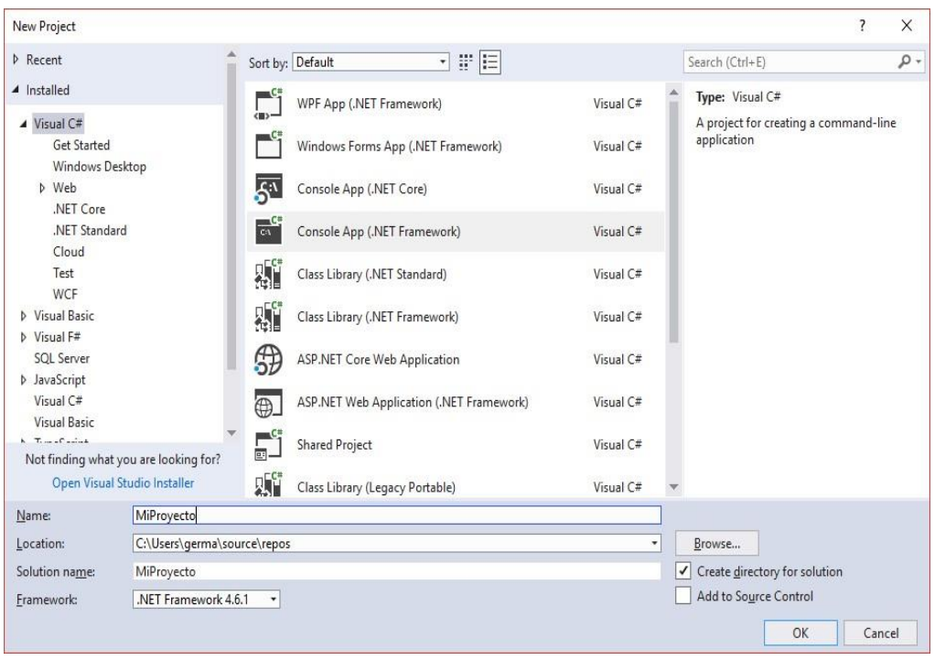
En la parte inferior encontramos otra ventana; en esta zona generalmente aparecen los apartados que utilizará el compilador para comunicarse con nosotros. Por ejemplo, aquí veremos la ventana que nos indica los errores que tiene nuestro programa, en la parte superior aparecen los menús y las barras de herramientas.

## Cómo crear nuestra primera aplicación

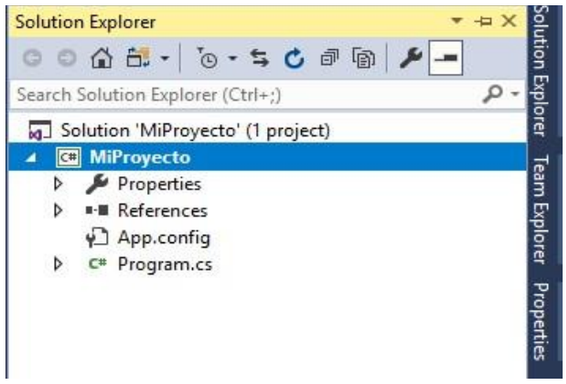
Para familiarizarnos más con el entorno, lo mejor es crear un primer proyecto y trabajar sobre él. Generaremos un pequeño programa que nos mande un mensaje en la consola, luego adicionaremos otros elementos para que podamos explorar las diferentes ventanas de la interfaz de usuario.

Para crear un proyecto tenemos que seleccionar el menú Archivo y luego de esto debemos hacer clic en la opción llamada Nuevo Proyecto, de esta forma veremos un cuadro de diálogo que muestra algunas opciones relacionadas.

En la parte central del cuadro de diálogo aparecen listados los diferentes tipos de proyectos que podemos crear. Para este ejemplo debemos seleccionar el que indica aplicación de consola. En la parte inferior escribiremos el nombre de nuestro proyecto, que en este caso será MiProyecto. Cada nuevo proyecto que crearemos tendrá su propio nombre; después, simplemente oprimimos el botón OK.



Visual Studio hace uso de soluciones y proyectos. Una solución puede tener varios proyectos; por ejemplo, Office tiene diferentes productos como Word, Excel y PowerPoint: Office es una solución y cada producto es un proyecto. El proyecto puede ser un programa independiente o una librería y puede tener uno o varios documentos. Estos documentos pueden ser el código fuente y recursos adicionales. Podemos observar que nuestra interfaz de usuario ha cambiado un poco, las ventanas ya nos muestran información y también podemos ver que en la zona de edición se muestra el esqueleto para nuestro programa.




El Explorador de soluciones nos muestra la información de la solución de forma lógica, siendo como un pequeño árbol. En la raíz encontramos a la solución. Cada proyecto que tengamos en esa solución será una rama. Cada proyecto, a su vez, tendrá también sus divisiones. En nuestro caso vemos tres elementos, en donde dos de ellos son carpetas; en una se guardan las propiedades del proyecto y en la otra las referencias (durante este libro no haremos uso de estas carpetas). El tercer elemento es un documento llamado Program.cs, el cual representa al documento donde guardamos el código fuente de nuestra aplicación. Vemos que la extensión de los programas de C# es .CS.

En el área de edición podemos observar que tenemos un esqueleto para que, a partir de ahí, podamos crear nuestro propio programa. Para entender lo que tenemos ahí es necesario conocer un concepto: namespace. El namespace es una agrupación lógica. Por ejemplo, todo el código que podemos tener relacionado con matemáticas puede quedar agrupado dentro del namespace de Math. Otro uso que tiene el namespace es el de resolver conflictos con los nombres. Por ejemplo: supongamos que tenemos un proyecto muy grande y varios programadores trabajando en él. Es posible que ambos programadores crearan un método que tuviera el mismo nombre, esto nos genera un conflicto ya que el programa no podría saber cuál versión utilizar. La forma de resolver esto es que cada programador tenga su propio namespace y hacer referencia al namespace correspondiente según la versión que deseáramos utilizar.

El Framework de .NET nos provee de varios namespaces donde tenemos miles de clases y métodos ya creados para nuestro uso. Cuando deseamos utilizar los recursos que se encuentran en un namespace programado por nosotros o por otros programadores, debemos hacer uso de un comando de C# conocido como using.

Como podemos ver en la parte superior del código, tenemos varios using haciendo referencia a los namespace que necesita nuestra aplicación; si necesitáramos adicionar más namespaces, lo haríamos en esta sección.

Más abajo se está definiendo el namespace propio de nuestro proyecto, esto se hace de la siguiente manera:

```
namespace MiProyecto

{

}
```

El namespace que estamos creando se llama MiProyecto. Como podemos ver, el namespace usa { } como delimitadores; esto se conoce como un bloque de código. Todo lo que se coloque entre { } pertenecerá al namespace; ahí es donde será necesario escribir el código correspondiente a nuestra aplicación.

Dentro del bloque de código encontramos la declaración de una clase. C# es un lenguaje orientado a objetos y por eso necesita que declaremos una clase para nuestra aplicación. La clase tiene su propio bloque de código y en nuestra aplicación se llamará Program.

Todos los programas necesitan de un punto de inicio, es decir, un lugar que indique a dónde empieza la ejecución del programa. En C#, al igual que en otros lenguajes, el punto de inicio es la función Main(), la cual también tiene su propio bloque de código. Dentro de esta función generalmente colocaremos el código principal de nuestra aplicación, aunque es posible tener más funciones o métodos y clases.

Ahora es el momento de crear nuestra primera aplicación. Vamos a modificar el código de la función tal y como se muestra a continuación. Cuando estemos adicionando la sentencia dentro de Main(), debemos fijarnos que sucede inmediatamente después de colocar el punto.

```
using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

namespace MiProyecto

{

    class Program

    {

        // Esta es la función principal del programa. Aquí inicia la
aplicación

static void Main(string[] args)

        {

            Console.WriteLine("Hola Mundo!");

        }

    }
```

```
}
```

Una vez que terminemos de escribir nuestro programa, podemos llevar a cabo la compilación. Como aprendimos anteriormente, esto va a generar el assembly que luego usará el runtime cuando se ejecute. Para poder compilar la aplicación debemos seleccionar el menú de Depuración o Debug y luego Construir Solución o Build Solution. El compilador empezará a trabajar y en la barra de estado veremos que nuestra solución se ha compilado exitosamente.

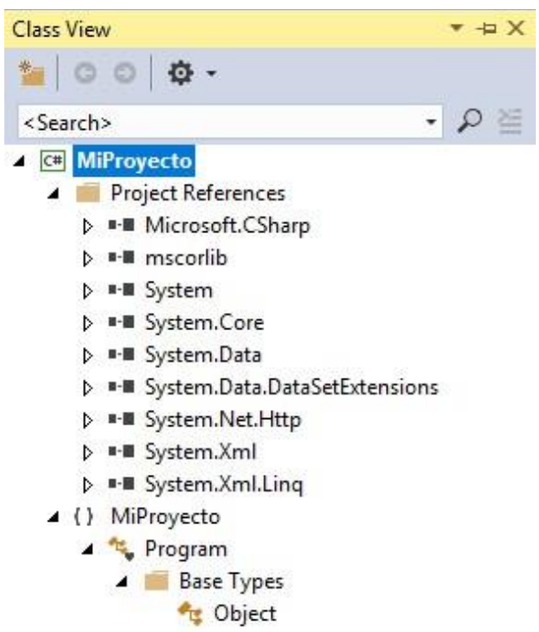
Una vez que la compilación ha sido exitosa, podemos ejecutar nuestro programa y ver cómo trabaja. Para esto tenemos dos opciones: ejecutar con depuración y ejecutar sin depurador. En la versión Express únicamente aparece la ejecución con depuración, pero podemos usar la ejecución sin depurador con las teclas CTRL+F5 o adicionándola, usando al menú de herramientas.

## La vista de clases

Ahora revisaremos la vista de clases. En esta vista podemos obtener información sobre la solución que estamos creando pero, a diferencia del Explorador de soluciones, la información se encuentra ordenada en forma lógica.

Con esta vista podemos encontrar rápidamente los namespace de nuestra solución y dentro de ellos las clases que contienen; si lo deseamos, podemos ver los métodos que se encuentran en cada clase. Esta vista no solamente permite observar la información lógica, también nos da la posibilidad de navegar rápidamente en nuestro código. En la versión Express no viene la opción previamente configurada, por lo que es necesario adicionar el comando al menú Vista.

Para mostrar la vista de clases debemos de ir al menú Vista y luego seleccionar Vista de Clases, una ventana aparecerá en nuestra interfaz de usuario.




La ventana de la vista de clases está divida en dos secciones, la sección superior nos muestra la relación lógica y jerárquica de los elementos mientras que en la parte inferior vemos los métodos que componen a alguna clase en particular.

En la raíz del árbol encontramos al proyecto, el cual va a contener las referencias necesarias y el namespace de MiProyecto. Si tuviéramos más namespaces estos aparecerían ahí, al abrir el namespace de MiProyecto encontramos las clases que están declaradas dentro de él. En este caso solamente tenemos la clase Program, si la seleccionamos veremos que en la parte inferior se muestran los elementos que estén declarados en su interior. En nuestro ejemplo tenemos el método Main().

Si en este momento damos doble clic en cualquiera de los métodos, nos dirigiremos automáticamente al código donde se define, como nuestro programa es muy pequeño, posiblemente no vemos la ventaja de esto, pero en programas con miles de líneas de código, el poder navegar rápidamente es una gran ventaja.

## Comentarios en el programa

Los programas de computadora pueden ser muy grandes, por lo que es necesario colocar comentarios en el código. Los comentarios sirven para documentar las partes del programa, explicar lo que hace cierta parte del código o simplemente colocar un recordatorio.

Podemos hacer comentarios de una sola línea, que se colocan por medio de la doble barra // seguido del texto del comentario. El texto que se encuentre a partir de la doble barra // y hasta el fin del renglón es ignorado por el compilador. Algunos ejemplos de comentarios pueden ser:

```
// Declaro mis variables
  int ancho, alto;
  int tarea; // Esta variable guarda el área calculada.
// La siguiente sentencia nunca se ejecuta
// Console.WriteLine(“Hola”);
```

Si llegáramos a necesitar colocar mucha información en nuestros comentarios, puede ser útil explayarla en varios renglones, es decir, hacer un bloque de comentarios. Para esto, abrimos nuestro bloque de comentarios con una barra seguida de un asterisco, /*, y finalizamos este con un asterisco en primer lugar, seguido de una barra */. Todo lo que se encuentre entre estos delimitadores será tomado como un comentario y el compilador lo pasará por alto. Veamos un ejemplo de un comentario de múltiples líneas:

```
/* La siguiente sección calcula el área y usa las variables
alto -para guardar la altura del rectángulo
ancho - para guardar la base del rectángulo
 */
```

## Las Variables

A las variables en la computadora las podemos imaginar cómo pequeñas cajas que existen en la memoria de la computadora y su tamaño dependerá de la información que deben

guardar. Esto se conoce como tipo de la variable. Para poder acceder a esa caja o variable le ponemos un nombre que sería la etiqueta con la que la identificamos.

Para hacer uso de una variable, lo primero que tenemos que hacer es declararla. La declaración de estas es algo muy sencillo. Como primer paso tenemos que colocar el tipo y luego el nombre. Las variables en C# deben nombrarse de acuerdo a unas recomendaciones sencillas:

- Los nombres de las variables deben empezar con letra.
- Es posible colocar números en los nombres de las variables, pero no empezar el nombre con ellos.
- Los nombres de las variables no pueden llevar signos a excepción del guion bajo.
- Las variables no pueden llevar acentos en sus nombres.

Cuando nombramos las variables, hay que tener en cuenta que C# es sensible a las mayúsculas y minúsculas, por lo que una variable llamada costo no es la misma que otra variable llamada COSTO u otra llamada Costo.

Es recomendable nombrar a las variables con nombres que hagan referencia a la información que guardarán. Si nos acostumbramos a hacer esto desde que empezamos a programar, evitaremos muchos problemas en el futuro y será mucho más sencillo corregir nuestros programas cuando tengan errores.

Veamos un ejemplo de cómo podemos declarar una variable que guarde valores numéricos enteros. El tipo que guarda estos valores se conoce como int.

```
int costo;
```

Vemos que al final de la sentencia hemos colocado el punto y coma. Si necesitamos declarar más variables lo podemos hacer de la siguiente forma:

```
int costo; int valor; int impuesto;
```

Pero también es posible declarar las variables en una sola línea. Para esto simplemente separamos los nombres de las variables con comas. No hay que olvidar colocar el punto y coma al final de la sentencia.

```
int costo, valor, impuesto;
```

C# nos provee de muchos tipos para poder usar con nuestras variables:

- **bool** es una variable booleana, es decir que solamente puede guardar los valores verdadero o falso (true o false) en términos de C#.
- **byte** puede guardar un byte de información. Esto equivale a un valor entero positivo entre 0 y 255.
- **sbyte** guarda un byte con signo de información. Podemos guardar un valor entero con signo desde –128 hasta 127.
- **char** puede guardar un carácter de tipo Unicode.
- **decimal** este tipo puede guardar un valor numérico con decimales. Su rango es desde ±1,0 x $10^{-28}$ to ±7,9228 x $10^{28}$
- **double** también nos permite guardar valores numéricos que tengan decimales. El rango aproximado es desde ±5.0 x $10^{-324}$ hasta ±1.7 x $10^{308}$.

- **float** otro tipo muy utilizado para guardar valores numéricos con decimales. Para este tipo el rango es desde ±1.5 x $10^{-45}$ hasta ±3.4 x $10^{38}$.
- **int** cuando queremos guardar valores numéricos enteros con signo, en el rango de -2,147,483,648 hasta 2,147,483,647.
- **uint** para valores numéricos enteros positivos, su rango de valores es desde 0 hasta 4,294,967,295.
- **long** guarda valores numéricos enteros realmente grandes con un rango desde –9,223,372,036,854,775,808 hasta 9,223,372,036,854,775,807.
- **ulong** guarda valores numéricos enteros positivos. Su rango de valores varía desde 0 hasta 18,446,744,073,709,551,615.
- **short** guarda valores numéricos enteros, pero su rango es menor y varía desde -32,768 hasta 32,767.
- **ushort** puede guardar valores numéricos enteros positivos con un rango desde 0 hasta 65,535.
- **string** este tipo nos permite guardar cadenas.

Una vez que hemos declarado la variable, debemos inicializarla. Inicializar una variable es asignarle un valor por primera vez. Esto nos permite darle un valor inicial que puede ser útil en la ejecución de nuestro programa (no solamente podemos asignarle a una variable un valor fijo, sino también puede ser desde un texto ingresado por el usuario o desde el registro de una base de datos).

Para asignarle un valor a una variable, ya sea durante la inicialización o durante el transcurso del programa, usamos un operador de asignación, el signo igual. Veamos a continuación un ejemplo de cómo asignar valores:

```
int costo, valor, impuesto;

costo = 50;

valor = 75;

impuesto = 10;
```

## Conversión de variables

Para poder convertir un dato numérico guardado como cadena a un valor numérico útil para nuestros propósitos, C# nos provee de una clase que permite hacer conversiones entre los diferentes tipos de variables. Esta clase se conoce como Convert y tiene una gran cantidad de métodos para la conversión. Aquí solamente veremos cómo convertir a enteros y flotantes, ya que son los tipos con los que trabajaremos. Sin embargo, los demás métodos se usan de forma similar y podemos encontrar la información necesaria sobre estos en MSDN (MicroSoft Developer Network).

Si queremos convertir del tipo string al tipo int usaremos el método ToInt32(). Este necesita un parámetro, que es la cadena que deseamos convertir. El método regresa un valor de tipo int, por lo que es necesario tener una variable que lo reciba.

Un ejemplo de esto es:

```
a = Convert.ToInt32(entrada);
```

De forma similar podemos convertir la cadena a una variable de tipo float. En este caso usaremos el método ToSingle(). Este también necesita un parámetro, que es la cadena que contiene el valor a convertir. Como regresa un valor flotante, entonces debemos tener una variable de tipo float que lo reciba. **valor = Convert.ToSingle(entrada);**

En el siguiente ejemplo vemos cómo utilizar la conversión de variables:

```
using System;

using System.Collections.Generic;

using System.Text;

namespace AplicacionBase

{

    class Program

    {

        // Esta es la función principal del programa

        // Aquí inicia la aplicación

        static void Main(string[] args)

        {

            // Declaramos variables

            string entrada = "";

            int a = 0, b = 0, resultado = 0;

            // Leemos una cadena

            Console.WriteLine("Escribe tu nombre");

            entrada = Console.ReadLine();

            Console.WriteLine("Hola {0}, como estas?", entrada);

            // Leemos dos valores y los sumamos.

            Console.Write("Dame un entero:");

            entrada = Console.ReadLine();

            // Convertimos la cadena a entero

            a = Convert.ToInt32(entrada);

            Console.Write("Dame otro numero entero:");

            entrada = Console.ReadLine();

            // Convertimos la cadena a entero
```

```
            b = Convert.ToInt32(entrada);
            // Sumamos los valores
            resultado = a + b;
            // Mostramos el resultado
            Console.WriteLine("El resultado es {0}", resultado);
        }
    }
}
```